

Statistical Computing

Methods for Generating Random Variables

Junyoung Byun

Spring 2026

Outline

Introduction

Inverse Transform Method

The Acceptance-Rejection Method

Transformation Methods

Sums and Mixtures

Multivariate Distributions

Introduction

Why generate random variables?

- A core tool in computational statistics is the ability to *simulate* random variables from specified probability distributions.
- Most methods for generating non-uniform random variables ultimately depend on a reliable generator for $\text{Uniform}(0, 1)$ pseudo-random numbers.
- In Python, we typically use NumPy's Generator:

$$U_1, \dots, U_n \stackrel{\text{iid}}{\sim} \text{Uniform}(0, 1) \iff \text{rng.random}(n). \quad (1)$$

Inverse Transform Method

Key result (continuous case)

If X is continuous with cdf F_X , then

$$U = F_X(X) \sim \text{Uniform}(0, 1). \quad (2)$$

Inverse transform idea

Define the generalized inverse

$$F_X^{-1}(u) = \inf\{x \in \mathbb{R} : F_X(x) \geq u\}, \quad 0 < u < 1. \quad (3)$$

If $U \sim \text{Uniform}(0, 1)$, then $F_X^{-1}(U)$ has cdf F_X .

Algorithm

1. Derive (or characterize) $F_X^{-1}(u)$.
 2. Implement a function that maps $u \mapsto F_X^{-1}(u)$.
 3. For each variate:
 - 3.1 Generate $u \sim \text{Uniform}(0, 1)$.
 - 3.2 Output $x = F_X^{-1}(u)$.
- Easy to apply when F_X^{-1} has a simple closed form.

Example: continuous inverse transform

Target density

Let $f_X(x) = 3x^2$ for $0 < x < 1$.

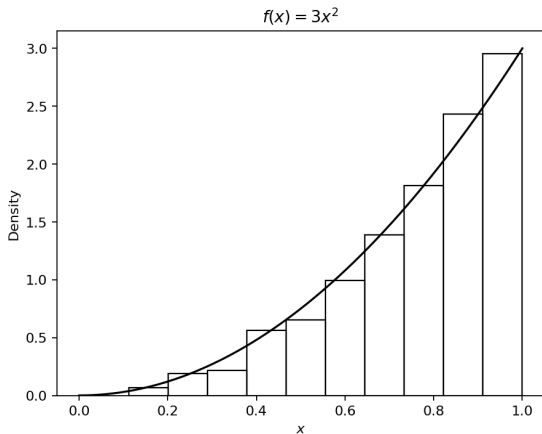
CDF and inverse

$$F_X(x) = \int_0^x 3t^2 dt = x^3, \quad 0 < x < 1, \quad (4)$$

so

$$F_X^{-1}(u) = u^{1/3}, \quad 0 < u < 1. \quad (5)$$

Python: sampling and checking by histogram



Probability density histogram of a random sample generated by the inverse transform method, with the theoretical density $f(x) = 3x^2$ superimposed.

Exponential distribution via inverse transform

CDF and inverse

If $X \sim \text{Exp}(\lambda)$, then for $x > 0$,

$$F_X(x) = 1 - e^{-\lambda x}, \quad F_X^{-1}(u) = -\frac{1}{\lambda} \log(1 - u). \quad (6)$$

Since U and $1 - U$ have the same distribution, one can use

$$X = -\frac{1}{\lambda} \log U. \quad (7)$$

Step-function CDF

For discrete X with jumps at $x_1 < x_2 < \dots$, the inverse is:

$$F_X^{-1}(u) = x_i \quad \text{if} \quad F_X(x_{i-1}) < u \leq F_X(x_i). \quad (8)$$

- Precompute a CDF array $(F(x_1), \dots, F(x_m))$ and find the first index where $F(x_i) \geq u$ (a search problem).

Example: Bernoulli via inverse CDF

Bernoulli(p)

$$\mathbb{P}(X = 1) = p, \quad \mathbb{P}(X = 0) = 1 - p. \quad (9)$$

Thus $F(0) = 1 - p$, $F(1) = 1$, and

$$X = \begin{cases} 0, & u \leq 1 - p, \\ 1, & u > 1 - p. \end{cases} \quad (10)$$

Example: geometric via inverse transform

Convention

Here geometric means the number of failures before the first success:

$$\mathbb{P}(X = x) = p(1 - p)^x, \quad x = 0, 1, 2, \dots \quad (11)$$

Inverse transform derivation

Let $q = 1 - p$. For integer $x \geq 0$,

$$F(x) = \mathbb{P}(X \leq x) = 1 - q^{x+1}. \quad (12)$$

Solving $F(x - 1) < u \leq F(x)$ yields the closed form:

$$X = \left\lceil \frac{\log U}{\log(1 - p)} \right\rceil - 1. \quad (13)$$

The Acceptance-Rejection Method

Assumption

Suppose X and Y have density/pmf f and g , respectively, and there exists a constant c such that

$$\frac{f(t)}{g(t)} \leq c \quad \text{for all } t \text{ such that } f(t) > 0. \quad (14)$$

- Use $Y \sim g$ to generate $X \sim f$.

Acceptance-Rejection Method: algorithm

Algorithm

1. Find a proposal Y with density/pmf g satisfying (14), and provide a method to generate Y .
2. For each random variate required:
 - 2.1 Generate y from g .
 - 2.2 Generate $u \sim \text{Uniform}(0, 1)$.
 - 2.3 If $u < \frac{f(y)}{cg(y)}$, accept and output $x = y$; otherwise reject and repeat.

Acceptance probability (given Y)

$$\mathbb{P}(\text{accept} \mid Y) = \mathbb{P}\left(u < \frac{f(Y)}{cg(Y)} \mid Y\right) = \frac{f(Y)}{cg(Y)}. \quad (15)$$

Total probability of acceptance (per iteration)

$$\mathbb{P}(\text{accept}) = \int \frac{f(y)}{cg(y)} g(y) dy = \frac{1}{c} \quad (\text{sum in the discrete case}). \quad (16)$$

- The number of iterations until acceptance has a geometric distribution with mean c . Hence, on average each sample value requires c iterations.
- For efficiency, Y should be easy to simulate and c should be small.

Why it works: accepted sample has distribution f

Bayes' Theorem (discrete case)

For each k such that $f(k) > 0$,

$$\begin{aligned}\mathbb{P}(k \mid \text{accepted}) &= \frac{\mathbb{P}(\text{accepted} \mid k) g(k)}{\mathbb{P}(\text{accepted})} \\ &= \frac{\left[\frac{f(k)}{c g(k)} \right] g(k)}{1/c} = f(k).\end{aligned}\tag{17}$$

- The continuous case is similar.

Example: Beta(2,2) via acceptance-rejection

Choice of g and c

The Beta(2,2) density is

$$f(x) = 6x(1 - x), \quad 0 < x < 1. \quad (18)$$

Let $g(x)$ be the Uniform(0,1) density. Then $f(x)/g(x) \leq 6$ for $0 < x < 1$, so we may take $c = 6$.

Acceptance test

A random x from g is accepted if

$$\frac{f(x)}{cg(x)} = \frac{6x(1 - x)}{6(1)} = x(1 - x) > u, \quad u \sim \text{Uniform}(0, 1). \quad (19)$$

Transformation Methods

- If we can express a target random variable as a transformation of simpler random variables, we get a generator “for free”.

Common transformations (1/2)

1. If $Z \sim N(0, 1)$, then $V = Z^2$ has a chi-square distribution with 1 degree of freedom.
2. If $U \sim \chi^2(m)$ and $V \sim \chi^2(n)$ are independent, then $F = (U/m)/(V/n)$ has an $F(m, n)$ distribution.
3. If $Z \sim N(0, 1)$ and $V \sim \chi^2(n)$ are independent, then $T = Z/\sqrt{V/n}$ has a $t(n)$ distribution.

Common transformations (2/2)

4. If $U, V \sim \text{Unif}(0, 1)$ independent, then

$$Z_1 = \sqrt{-2 \log U} \cos(2\pi V), \quad Z_2 = \sqrt{-2 \log U} \sin(2\pi V) \quad (20)$$

are independent $N(0, 1)$.

5. If $U \sim \Gamma(r, \lambda)$ and $V \sim \Gamma(s, \lambda)$ are independent, then $X = U/(U + V)$ is $\text{Beta}(r, s)$.
6. If $U, V \sim \text{Unif}(0, 1)$ are independent, then

$$X = \left\lceil 1 + \frac{\log(V)}{\log(1 - (1 - \theta)^U)} \right\rceil \quad (21)$$

has the $\text{Logarithmic}(\theta)$ distribution.

Example: Beta via ratio of Gammas

Gamma-to-Beta transformation

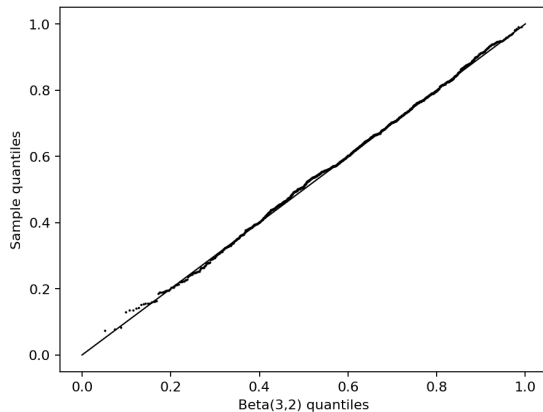
Let $U \sim \Gamma(a, 1)$ and $V \sim \Gamma(b, 1)$ be independent. Then

$$X = \frac{U}{U + V} \sim \text{Beta}(a, b). \quad (22)$$

Algorithm

1. Generate $u \sim \Gamma(a, 1)$.
2. Generate $v \sim \Gamma(b, 1)$.
3. Output $x = u / (u + v)$.

Python: QQ plot diagnostic for Beta(3,2)



QQ plot comparing a Beta(3,2) distribution with a simulated sample generated by the ratio-of-gammas method.

Sums and Mixtures

Convolutions: sums of independent random variables

Definition

Let $X_1, \dots, X_n$ be independent and identically distributed (i.i.d.) with the same distribution as X . Define the sum

$$S = X_1 + \dots + X_n. \quad (23)$$

The distribution of S is the n -fold convolution of the distribution of X .

- To simulate a convolution, simulate $X_1, \dots, X_n$ and return the sum S .
- Common examples:
 - $\chi^2(\nu)$ as a sum of ν squared standard normals
 - $\Gamma(r, \lambda)$ as a sum of r i.i.d. exponential variates.

Mixtures: discrete and continuous

Discrete mixture

A random variable X is a discrete mixture if its cdf can be written as

$$F_X(x) = \sum_i w_i F_{X_i}(x), \quad w_i > 0, \quad \sum_i w_i = 1, \quad (24)$$

where the w_i are *mixing weights* and X_i are component variables.

Continuous mixture

A continuous mixture can be expressed as

$$F_X(x) = \int_{-\infty}^{\infty} F_{X|Y=y}(x) f_Y(y) dy, \quad \int f_Y(y) dy = 1. \quad (25)$$

Convolution vs mixture: what changes?

Convolution (sum)

Generate *all* components and add them:

$$S = X_1 + X_2. \quad (26)$$

For example, if $X_1 \sim N(0, 1)$ and $X_2 \sim N(3, 1)$ independent, then S is normal with mean 3 and variance 2.

Mixture (choose-one)

Randomly choose a component index K and draw from that component:

$$X \sim \begin{cases} X_1, & K = 1, \\ X_2, & K = 2, \end{cases} \quad \mathbb{P}(K = 1) = \mathbb{P}(K = 2) = \frac{1}{2}. \quad (27)$$

Even when X_1, X_2 are normal, the mixture is generally *not* normal (often bimodal).

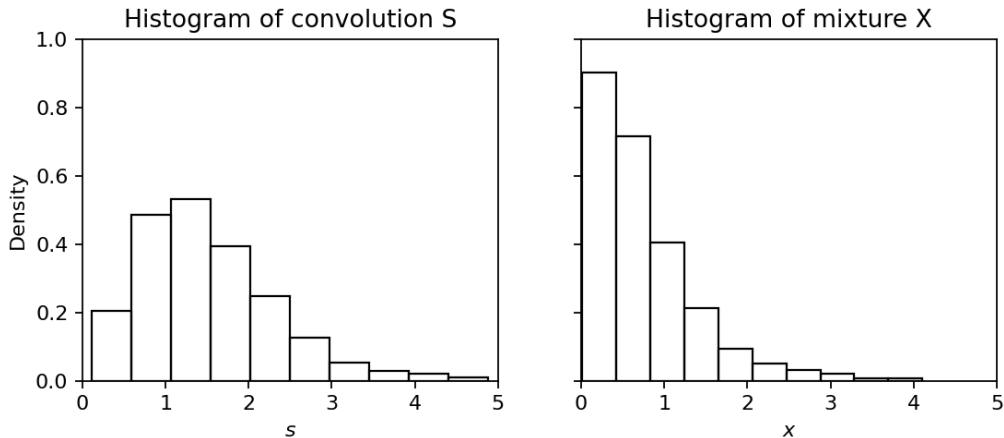
Example: convolution vs mixture (Gamma)

Setup

Let $X_1 \sim \Gamma(2, 2)$ and $X_2 \sim \Gamma(2, 4)$ be independent. Compare:

$$S = X_1 + X_2 \quad (\text{convolution}), \quad F_X = \frac{1}{2}F_{X_1} + \frac{1}{2}F_{X_2} \quad (\text{mixture}). \quad (28)$$

Python: histogram comparison (two panels)



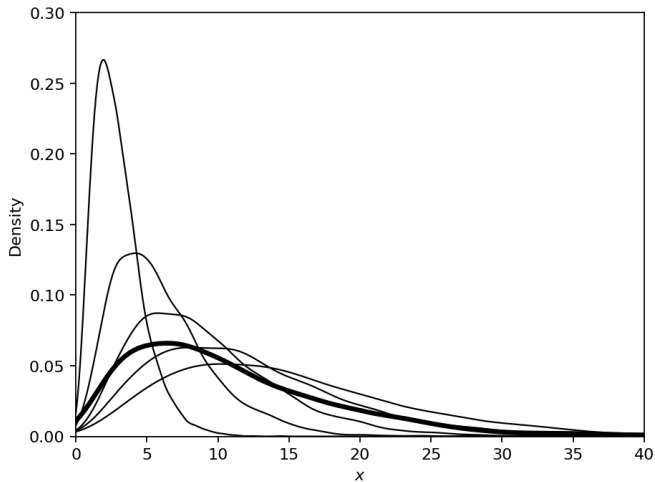
Histogram comparison of a convolution $S = X_1 + X_2$ and a 50% mixture of the same gamma components.

General recipe

To simulate a discrete mixture with components $\{F_j\}_{j=1}^m$ and weights $\{w_j\}$:

1. Sample indices $K_1, \dots, K_n \in \{1, \dots, m\}$ with $\mathbb{P}(K = i) = w_i$.
2. For each i , sample $X_i \sim F_{K_i}$.

Python: plot KDE of mixture and components



Density estimates of a gamma mixture (thick line) and its component densities (thin lines).

Multivariate Distributions

Multivariate Normal (MVN): definition

Density

A random vector $X = (X_1, \dots, X_d)^T$ follows a d -dimensional multivariate normal distribution, denoted $N_d(\mu, \Sigma)$, if it has density

$$f(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right), \quad x \in \mathbb{R}^d, \quad (29)$$

where $\mu \in \mathbb{R}^d$ and Σ is symmetric positive definite.

Covariance matrix

$$\Sigma = (\sigma_{ij})_{i,j=1}^d, \quad \sigma_{ij} = \text{Cov}(X_i, X_j). \quad (30)$$

Key fact

If $Z \sim N_d(0, I_d)$ and C is a $d \times d$ matrix, then

$$CZ + \mu \sim N_d(\mu, CC^T). \quad (31)$$

Therefore, if we can factor the covariance matrix as

$$\Sigma = Q^T Q, \quad (32)$$

then

$$X = Q^T Z + \mu \sim N_d(\mu, \Sigma). \quad (33)$$

Sample (matrix) form

Let Z be an $n \times d$ matrix with i.i.d. $N(0, 1)$ entries (each row is one Z -vector). Then the transformed sample matrix

$$X = ZQ + \mathbf{1}\mu^T \quad (34)$$

has rows i.i.d. from $N_d(\mu, \Sigma)$.

Algorithm: generating $N_d(\mu, \Sigma)$ samples

Procedure

To generate n samples from $N_d(\mu, \Sigma)$:

1. Generate an $n \times d$ matrix Z of i.i.d. $N(0, 1)$ variates.
2. Compute a factorization $\Sigma = Q^T Q$.
3. Transform: $X = ZQ + \mathbf{1}\mu^T$.
4. Output X (each row is one draw).

Common choices:

- eigen-decomposition (spectral decomposition),
- singular value decomposition (SVD),
- Cholesky factorization.

Square root of covariance

For symmetric positive definite Σ , an eigen-decomposition gives

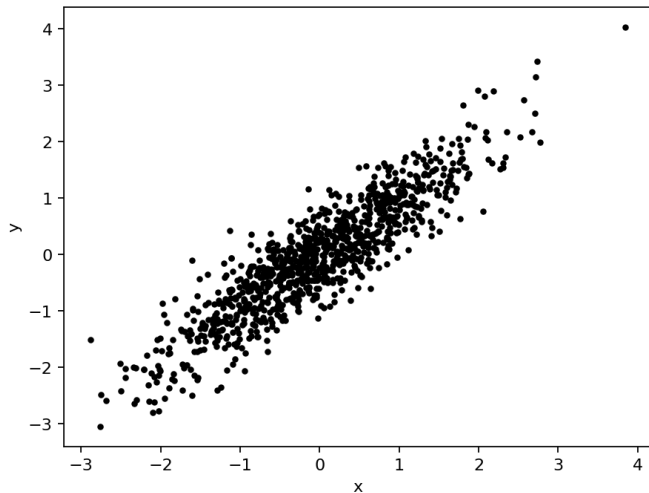
$$\Sigma = V\Lambda V^T, \quad (35)$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ and V has orthonormal columns. Define

$$Q = V\Lambda^{1/2}V^T. \quad (36)$$

Then Q is symmetric and satisfies $Q^T Q = \Sigma$.

Example: bivariate normal check (mean/correlation)



Scatter plot of a simulated bivariate normal sample.

SVD method (for symmetric positive definite Σ)

SVD and covariance square root

The singular value decomposition is $\Sigma = UDV^T$. For symmetric positive definite Σ , we have $U = V$ and the singular values match eigenvalues. A valid square root is

$$Q = UD^{1/2}V^T. \quad (37)$$

Then $Q^T Q = \Sigma$.

- For symmetric positive definite matrices, the SVD method is typically less efficient than eigen/Cholesky because it does not fully exploit symmetry.

Cholesky factor

For symmetric positive definite Σ , the Cholesky factorization gives

$$\Sigma = LL^T, \quad (38)$$

where L is lower triangular. If we set $Q = L^T$, then $Q^T Q = \Sigma$.

Two-component MVN mixture

A two-component multivariate normal mixture can be written as

$$X \sim p N_d(\mu_1, \Sigma_1) + (1 - p) N_d(\mu_2, \Sigma_2), \quad 0 < p < 1. \quad (39)$$

With probability p , the population is $N_d(\mu_1, \Sigma_1)$; otherwise it is $N_d(\mu_2, \Sigma_2)$.

1. Generate $U \sim \text{Uniform}(0, 1)$ (or $K \sim \text{Bernoulli}(p)$).
2. If $U \leq p$ (or $K = 1$), sample from component 1; otherwise component 2.

Example: 4D 50% location mixture

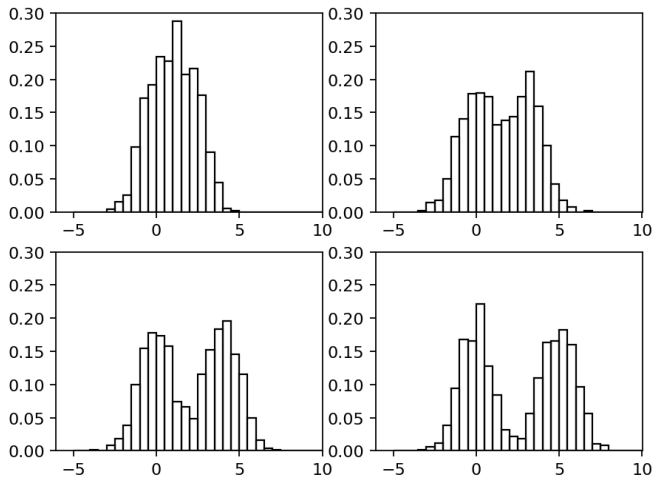
Setup

Generate $n = 1000$ observations from a 4-dimensional location mixture:

$$p = \frac{1}{2}, \quad \mu_1 = (0, 0, 0, 0), \quad \mu_2 = (2, 3, 4, 5), \quad \Sigma = I_4. \quad (40)$$

- Since 4D data are hard to visualize directly, we plot histograms of the 1D marginals.

Python: 4D location mixture + marginal histograms



Histograms of the marginal distributions of a multivariate normal location mixture.

Wishart distribution: definition

Definition (sample cross-product)

Let X be an $n \times d$ data matrix whose rows are an i.i.d. sample from $N_d(\mu, \Sigma)$. Define

$$M = X^T X. \quad (41)$$

Then M follows a Wishart distribution with scale matrix Σ and n degrees of freedom, denoted

$$M \sim W_d(\Sigma, n). \quad (42)$$

- Wishart random matrices arise naturally as random covariance/scatter matrices in multivariate statistics.

Bartlett decomposition for Wishart

Bartlett's decomposition

Let $T = (T_{ij})$ be a lower triangular $d \times d$ random matrix with independent entries:

1. $T_{ij} \sim N(0, 1)$ for $i > j$.
2. $T_{ii} \sim \sqrt{\chi^2(n - i + 1)}$ for $i = 1, \dots, d$ (with n degrees of freedom).

Then

$$A = TT^T \sim W_d(I_d, n). \quad (43)$$

General scale matrix

If $\Sigma = LL^T$ is the Cholesky factorization (lower triangular L), then

$$W = LAL^T \sim W_d(\Sigma, n). \quad (44)$$

- A direct approach is to simulate an $n \times d$ MVN sample and compute $X^T X$. This is simple but can be computationally expensive because it generates nd normals to produce only $d(d+1)/2$ distinct entries.
- Bartlett's decomposition generates the Wishart matrix more efficiently by exploiting triangular structure.

Uniform distribution on the d -sphere

The unit sphere

The d -sphere is

$$\mathbb{S}^{d-1} = \{x \in \mathbb{R}^d : \|x\| = (x^T x)^{1/2} = 1\}. \quad (45)$$

Key property

If $X_1, \dots, X_d$ are i.i.d. $N(0, 1)$ and $X = (X_1, \dots, X_d)$, then

$$U = \frac{X}{\|X\|} \quad (46)$$

is uniformly distributed on the unit sphere.

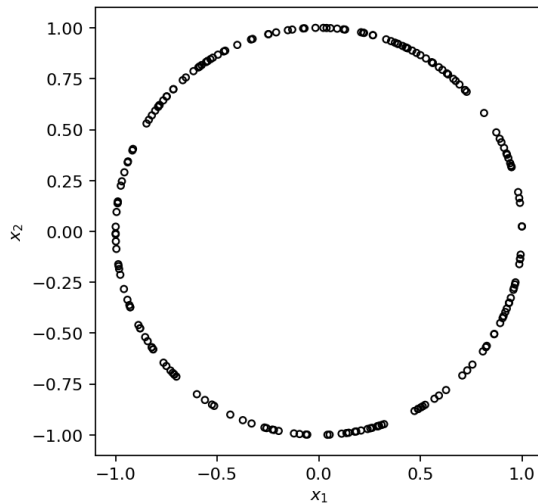
Algorithm: uniform points on the sphere

Algorithm

To generate n points uniformly on S^{d-1} :

1. Generate an $n \times d$ matrix M with i.i.d. $N(0, 1)$ entries.
2. Compute row norms $L_i = \|M_{i,\cdot}\|$ for $i = 1, \dots, n$.
3. Normalize each row: $U_{i,\cdot} = M_{i,\cdot} / L_i$.

Python: 2D unit circle plot



A random sample of points uniformly distributed on the unit circle in $\mathbb{R}^2$.

- Univariate generators: inverse transform, rejection sampling, and transformations.
- Convolutions and mixtures: sums of independent variables vs. choose-one mixtures.
- Multivariate normals: generation via covariance factorization.
- MVN mixtures: flexible departures from normality (e.g., multimodality, skewness).
- Wishart matrices: random covariance/scatter matrices via Bartlett decomposition.
- Uniform directions: uniform sampling on the d -sphere via normalization of normals.